

CS 5151 / 6051

Database Theory (DBT)

06: Advanced SQL

Seokki Lee

Slides are adapted from:

Database System Concepts, 6th & 7th Ed. ©Silberschatz, Korth and Sudarshan

Outline

- **Functions and Procedures**
- Triggers
- Recursive Queries
- Advanced Aggregation Features
- Accessing SQL From a Programming Language

Functions and Procedures

- What are functions and procedures?

Functions and Procedures

- Allow “business logic” to be stored in the database and executed from SQL statements.
 - How many courses a student can take in a given semester
 - Minimum number of courses a full-time instructor must teach per year
 - Maximum number of majors a student can be enrolled in
 - ...
- SQL allows to define functions and procedures by the procedural component of it.
 - Standard syntax here differs from procedural languages, e.g., Oracle (PL/SQL), PostgreSQL (PL/pgSQL), etc.

Declaring SQL Functions

- What does this function define?

```
create function dept_count (dept_name varchar(20))  
returns integer  
begin  
  declare d_count integer;  
    select count ( * ) into d_count  
    from instructor  
    where instructor.dept_name = dept_name  
  return d_count;  
end
```

Declaring SQL Functions

- Define a function that, given the name of a department, returns the count of the number of instructors in that department.

```
create function dept_count (dept_name varchar(20))  
  returns integer  
  begin  
    declare d_count integer;  
    select count ( * ) into d_count  
    from instructor  
    where instructor.dept_name = dept_name  
  return d_count;  
  end
```

Declaring SQL Functions

- Define a function that, given the name of a department, returns the count of the number of instructors in that department.

```
create function dept_count (dept_name varchar(20))  
  returns integer  
  begin  
    declare d_count integer;  
    select count ( * ) into d_count  
    from instructor  
    where instructor.dept_name = dept_name;  
    return d_count;  
  end
```

- The function *dept_count* can be used to find the department names and budget of all departments with more than 12 instructors.

```
select dept_name, budget  
from department  
where dept_count (dept_name) > 12
```

Table Functions

- The SQL standard supports functions that can return tables as results.
- Example: Return all instructors in a given department

```
create function instructor_of (dept_name varchar(20))
```

```
returns table (
```

```
    ID varchar(5),  
    name varchar(20),  
    dept_name varchar(20),  
    salary numeric(8,2))
```

```
return table
```

```
    (select ID, name, dept_name, salary  
    from instructor  
    where instructor.dept_name = instructor_of.dept_name)
```

Table Functions

- The SQL standard supports functions that can return tables as results.
- Example: Return all instructors in a given department

```
create function instructor_of (dept_name varchar(20))
```

```
returns table (
```

```
    ID varchar(5),  
    name varchar(20),  
    dept_name varchar(20),  
    salary numeric(8,2))
```

```
return table
```

```
    (select ID, name, dept_name, salary  
    from instructor  
    where instructor.dept_name = instructor_of.dept_name)
```

- Usage

```
select *  
from table (instructor_of ('Music'))
```

Are Functions Useful?

Are Functions Useful?

- Functions are particularly useful with specialized data types
 - Images and geometric objects
 - A **line-segment data** type used in a map database may have an associated function that checks whether two line segments overlap
 - An **image data** type may have associated functions to compare two images for similarity.

SQL Procedures

- The keywords **in** and **out** are parameters that are expected to have values assigned to them and parameters whose values are set in the procedure in order to return results.
- The *dept_count* function could instead be written as procedure:

```
create procedure dept_count_proc (in dept_name varchar(20),  
                                out d_count integer)  
begin  
    select count(*) into d_count  
    from instructor  
    where instructor.dept_name = dept_count_proc.dept_name  
end
```

SQL Procedures

- The keywords **in** and **out** are parameters that are expected to have values assigned to them and parameters whose values are set in the procedure in order to return results.
- The *dept_count* function could instead be written as procedure:

```
create procedure dept_count_proc (in dept_name varchar(20),  
                                out d_count integer)
```

```
begin
```

```
    select count(*) into d_count
```

```
    from instructor
```

```
    where instructor.dept_name = dept_count_proc.dept_name
```

```
end
```

- Procedures can be invoked either from an SQL procedure or from embedded SQL, using the **call** statement.

```
    declare d_count integer;
```

```
    call dept_count_proc( 'Physics', d_count );
```

SQL Procedures

- Procedures and functions can be invoked also from dynamic SQL.
- SQL allows more than one procedure of the so long as the number of arguments of the procedures with the same name is different.
 - The name, along with the number of arguments, is used to identify the procedure.

Language Constructs for Procedures & Functions

- SQL supports constructs that gives it almost all the power of a general-purpose programming language.
 - Warning: most database systems implement their own variant of the standard syntax below.
- Variables: **declare**
 - Can have any valid SQL data type
- Compound statement: **begin ... end**,
 - May contain multiple SQL statements between **begin** and **end**.
 - Local variables can be declared within a compound statements.
 - **begin atomic ... end** ensures all statements are executed as a single transaction.

Language Constructs for Procedures & Functions

- While and repeat statements:
 - **while** boolean expression **do**
 sequence of statements ;
end while
 - **repeat**
 sequence of statements ;
until boolean expression
end repeat

```
declare  $n$  integer default 0;  
while  $n < 10$  do  
    set  $n = n + 1$   
end while  
  
repeat  
    set  $n = n - 1$   
until  $n = 0$   
end repeat
```

Language Constructs (Cont.)

- **For** loop
 - Permits iteration over all results of a query
- Example: Find the budget of music department

```
declare n integer default 0;  
for r as  
    select budget from department  
    where dept_name = 'Music'  
do  
    set n = n - r.budget  
end for
```

Language Constructs – if-then-else

- Conditional statements (**if-then-else**)

if *boolean expression*
 then *statement or compound statement*
elseif *boolean expression*
 then *statement or compound statement*
else *statement or compound statement*
end if

Example procedure

- Registers student after ensuring classroom capacity is not exceeded
 - Returns 0 on success and -1 if capacity is exceeded

```
begin
  declare currEnrol int;
  select count(*) into currEnrol
    from takes
   where course_id = s_courseid and sec_id = s_secid
      and semester = s_semester and year = s_year;
  declare limit int;
  select capacity into limit
    from classroom natural join section
   where course_id = s_courseid and sec_id = s_secid
      and semester = s_semester and year = s_year;
  if (currEnrol < limit)
    begin
      insert into takes values
        (s_id, s_courseid, s_secid, s_semester, s_year, null);
      return(0);
    end
  -- Otherwise, section capacity limit already reached
  set errorMsg = 'Enrollment limit reached for course ' || s_courseid
    || ' section ' || s_secid;
  return(-1);
end;
```

Example procedure

- Signaling of **exception conditions**, and declaring **handlers** for exceptions

```
declare out_of_classroom_seats condition  
declare exit handler for out_of_classroom_seats  
begin  
...  
end
```

- The statements between the **begin** and the **end** can raise an exception by executing “**signal** *out_of_classroom_seats*”
- The handler says that if the condition arises, the action to be taken is to exit the enclosing the **begin...end** statement.

Functions and Procedures

- These can be defined either by the procedural component of SQL or by an external programming language such as Java, C, or C++.
 - Advantage of not being outside of database?

Functions and Procedures

- These can be defined either by the procedural component of SQL or by an external programming language such as Java, C, or C++.
 - Multiple applications to access the procedures
 - A single point change (without changing other parts of the application)

Pros and Cons

Pros and Cons

- Pros
 - Efficient: precompiled and cached
 - Reusable: multiple applications and users
 - Secure

- Cons
 - Not easy to debug
 - Cost
 - Versioning

Difference: Functions and Procedures

Stored Procedures

Functions

Difference: Functions and Procedures

Stored Procedures	Functions
Returning zero or a single value or multiple values	Returning a single value or a set
Input and output parameters	Only input parameters
Cannot be used in select, where, having clause	Can be used in select, where, having clause
Cannot be called from functions	Can be called from procedure
Transaction management	-
Exception handling	-

External Language Routines

- SQL allows us to define functions in a programming language such as Java, C#, C or C++.
- Declaring external language procedures and functions

```
create procedure dept_count_proc(in dept_name varchar(20),  
                                out count integer)
```

```
language C
```

```
external name '/usr/avi/bin/dept_count_proc'
```

```
create function dept_count(dept_name varchar(20))
```

```
returns integer
```

```
language C
```

```
external name '/usr/avi/bin/dept_count'
```

External Language Routines (Cont.)

- Benefits of external language functions/procedures:

External Language Routines (Cont.)

- Benefits of external language functions/procedures:
 - More **efficient** for many operations
 - More **expressive** power

External Language Routines (Cont.)

- Benefits of external language functions/procedures:
 - More **efficient** for many operations
 - More **expressive** power
- Drawbacks

External Language Routines (Cont.)

- Benefits of external language functions/procedures:
 - More **efficient** for many operations
 - More **expressive** power
- Drawbacks
 - Code to implement function may need to be loaded into database system and executed in the database system's address space.
 - Risk of accidental corruption of database structures
 - Security risk, allowing users access to unauthorized data
 - There are alternatives, which give good security at the cost of potentially worse performance.

Security with External Language Routines

- To deal with security problems, we can do one of the following:
 - Use **sandbox** techniques
 - That is, use a safe language like Java or C#, which cannot be used to access/damage other parts of the database code.
 - Allow the code to access its own memory area
 - Run external language functions/procedures in a **separate process**, with no access to the database process' memory.
 - Parameters and results communicated via inter-process communication
- Both have performance **overheads**.
- Many database systems support both above approaches as well as direct executing in database system address space.

Outline

- Functions and Procedures
- **Triggers**
- Recursive Queries
- Advanced Aggregation Features
- Accessing SQL From a Programming Language

Triggers

- A **trigger** is a statement that is executed automatically by the system as a side effect of a modification to the database.

Triggers

- A **trigger** is a statement that is executed automatically by the system as a side effect of a modification to the database.
- To design a trigger mechanism, we must:
 - Specify the **conditions** under which the trigger is to be executed.
 - Specify the **actions** to be taken when the trigger executes.

Triggers

- A **trigger** is a statement that is executed automatically by the system as a side effect of a modification to the database.
- To design a trigger mechanism, we must:
 - Specify the **conditions** under which the trigger is to be executed.
 - Specify the **actions** to be taken when the trigger executes.
- Usecase
 - To impement certain integrity constraints that can't be specified using constraint mechanism
 - For starting certain tasks automatically when certain conditions are met

Example of a Trigger

- Example: using a trigger to maintain referential integrity
 - Initiated after any insertion on relation *section*
 - Ensuring the *time_slot_id* value is valid

section(*course_id*, *sec_id*, *semester*, *year*, *building*, *room_number*, *time_slot_id*)
time_slot(*time_slot_id*, *day*, *start_time*, *end_time*)

create trigger *timeslot_check1* after insert on *section*

Initiated after the
insertion on *section*

referencing new row as *nrow*

Create a variable storing the inserted row

for each row

Iterate each inserted row

when (*nrow.time_slot_id* not in (

select *time_slot_id*

from *time_slot*)) /* *time_slot_id* not present in *time_slot* */

begin

rollback

end;

Triggering Events and Actions in SQL

- Triggering event can be **insert**, **delete** or **update**
- Triggers on update can be restricted to specific attributes
 - **after update of** *takes on grade*
 - Updates to other attributes would not cause it to be executed.
- Values of attributes before and after an update can be referenced
 - **referencing old row as** : for deletes and updates
 - **referencing new row as** : for inserts and updates

Triggering Events and Actions in SQL

- Triggers can be activated before an event, which can serve as extra constraints.
 - Attempt to insert an instructor into a new department
 - Referential integrity
 - The department does not exist in *department*.

Instructor

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

department

<i>dept_name</i>	<i>building</i>	<i>budget</i>
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000

Triggering Events and Actions in SQL

- Triggers can be activated before an event, which can serve as extra constraints.
 - What does below trigger do?

```
create trigger setnull_trigger before update of takes  
referencing new row as nrow  
for each row  
when (nrow.grade = ' ' )  
begin atomic  
    set nrow.grade = null;  
end;
```

Triggering Events and Actions in SQL

- Triggers can be activated before an event, which can serve as extra constraints.
 - Convert blank grades to null

```
create trigger setnull_trigger before update of takes  
referencing new row as nrow  
for each row  
when (nrow.grade = ' ' )  
begin atomic  
    set nrow.grade = null;  
end;
```

Trigger to Maintain credits_earned value

- Define a trigger to keep the tot_cred attribute value of student tuples up-to-date
- Condition: Executed only after the grade value updated
 - From either null or F
 - With other grades that indicate successful completion
- Make the transaction atomic

course(course_id, title, dept_name, credits)
student(ID, name, dept_name, tot_cred)
takes(ID, course_id, sec_id, semester, year, grade)

Trigger to Maintain `credits_earned` value

- create trigger *credits_earned* after update of *takes* on *grade*

course(*course_id*, *title*, *dept_name*, *credits*)
student(*ID*, *name*, *dept_name*, *tot_cred*)
takes(*ID*, *course_id*, *sec_id*, *semester*, *year*, *grade*)

Trigger to Maintain `credits_earned` value

- create trigger *credits_earned* after update of *takes* on *grade*
referencing new row as *nrow*
referencing old row as *orow*
for each row

```
course(course_id, title, dept_name, credits)
student(ID, name, dept_name, tot_cred)
takes(ID, course_id, sec_id, semester, year, grade)
```

Trigger to Maintain `credits_earned` value

- **create trigger *credits_earned* after update of *takes* on *grade***
referencing new row as *nrow*
referencing old row as *orow*
for each row
when *nrow.grade* $\neq$ 'F' and *nrow.grade* is not null
and (*orow.grade* = 'F' or *orow.grade* is null)

```
course(course_id, title, dept_name, credits)  
student(ID, name, dept_name, tot_cred)  
takes(ID, course_id, sec_id, semester, year, grade)
```

Trigger to Maintain `credits_earned` value

- **create trigger *credits_earned* after update of *takes* on *grade***
referencing new row as *nrow*
referencing old row as *orow*
for each row
when *nrow.grade* $\neq$ 'F' and *nrow.grade* is not null
and (*orow.grade* = 'F' or *orow.grade* is null)
begin atomic
update *student*
set *tot_cred* = *tot_cred* +
(select *credits*
from *course*
where *course.course_id* = *nrow.course_id*)
where *student.id* = *nrow.id*;
end;

course(*course_id*, *title*, *dept_name*, *credits*)
student(*ID*, *name*, *dept_name*, *tot_cred*)
takes(*ID*, *course_id*, *sec_id*, *semester*, *year*, *grade*)

Statement Level Triggers

- Instead of executing a separate action for each affected row, a single action can be executed for all rows affected by a transaction.
 - **for each statement** instead of **for each row**

Statement Level Triggers

- Instead of executing a separate action for each affected row, a single action can be executed for all rows affected by a transaction.
 - **for each statement** instead of **for each row**
 - **referencing old table** or **referencing new table** to refer to temporary tables (called *transition tables*) containing the affected rows
 - Can be more efficient when dealing with SQL statements that update a large number of rows

When Not To Use Triggers

- Triggers were used earlier for tasks such as
 - Maintaining summary data (e.g., total salary of each department)
 - Replicating databases by recording changes to special relations (called **change** or **delta** relations) and having a separate process that applies the changes over to a replica
- Example view definition

section_registration(course_id, sec_id, semester, year, total_students)

select *course_id, sec_id, semester, year, count(ID) as total_students*
from *takes*
group by *course_id, sec_id, semester, year;*

When Not To Use Triggers (Cont.)

- There are better ways of doing these now:
 - Databases today provide built in materialized view facilities to maintain summary data automatically.
 - Databases provide built-in support for replication.
- Encapsulation facilities can be used instead of triggers in many cases
 - Define methods to update fields
 - Carry out actions as part of the update methods instead of through a trigger

When Not To Use Triggers (Cont.)

- Risk of unintended execution of triggers, for example, when
 - Loading data from a backup copy
 - Replicating updates at a remote site
 - Trigger execution can be disabled before such actions.
- Other risks with triggers:
 - Error leading to failure of critical transactions that set off the trigger
 - Cascading execution

Outline

- Functions and Procedures
- Triggers
- **Recursive Queries**
- Advanced Aggregation Features
- Accessing SQL From a Programming Language

Recursion in SQL

- SQL:1999 permits recursive view definition.
- Example: find which courses are a prerequisite, whether directly or indirectly, for a specific course

<i>course_id</i>	<i>prereq_id</i>
BIO-301	BIO-101
BIO-399	BIO-101
CS-190	CS-101
CS-315	CS-190
CS-319	CS-101
CS-319	CS-315
CS-347	CS-319

Recursion in SQL - Syntax

- General form
with recursive *R* **as** (
 init_query
 union
 recursive_step)
select *
from *R*;
- init_query returns the initial content of R
 - Definition of R
- recursive_step is a query that mentions R exactly once in the FROM clause
 - Query involving R

Recursion in SQL - Semantics

- General form
with recursive R **as** (
 init_query
 union
 recursive_step)
select *
from R ;
- **Fixed** point computation
 - R_0 = result of init_query
 - In step i : R_i is computed as
 - $R_{i-1} \text{ union recursive_step}(R_{i-1})$
 - The computation stops when recursive_step(R_{i-1}) is the empty set, i.e., $R_{i-1} = R_i$

Recursion in SQL

- Example: find which courses are a prerequisite, whether directly or indirectly, for a specific course
 - **with recursive R as** (
 init_query
 union
 recursive_step)
select *
from R ;

<i>course_id</i>	<i>prereq_id</i>
BIO-301	BIO-101
BIO-399	BIO-101
CS-190	CS-101
CS-315	CS-190
CS-319	CS-101
CS-319	CS-315
CS-347	CS-319

Recursion in SQL

- Example: find which courses are a prerequisite, whether directly or indirectly, for a specific course

- **with recursive** *rec_prereq(course_id, prereq_id)* **as** (
 select *course_id, prereq_id*
 from *prereq*
 union
 select *rec_prereq.course_id, prereq.prereq_id,*
 from *rec_prereq, prereq*
 where *rec_prereq.prereq_id = prereq.course_id*
)
 select *
from *rec_prereq*;

<i>course_id</i>	<i>prereq_id</i>
BIO-301	BIO-101
BIO-399	BIO-101
CS-190	CS-101
CS-315	CS-190
CS-319	CS-101
CS-319	CS-315
CS-347	CS-319

Recursion in SQL

- Example: find which courses are a prerequisite, whether directly or indirectly, for a specific course

- with recursive** *rec_prereq(course_id, prereq_id)* **as** (
 select *course_id, prereq_id*
 from *prereq*
 union
 select *rec_prereq.course_id, prereq.prereq_id,*
 from *rec_prereq, prereq*
 where *rec_prereq.prereq_id = prereq.course_id*
)
select *
from *rec_prereq*;

<i>course_id</i>	<i>prereq_id</i>
BIO-301	BIO-101
BIO-399	BIO-101
CS-190	CS-101
CS-315	CS-190
CS-319	CS-101
CS-319	CS-315
CS-347	CS-319

<i>Iteration Number</i>	<i>Tuples in c1</i>
0	
1	(CS-319)
2	(CS-319), (CS-315), (CS-101)
3	(CS-319), (CS-315), (CS-101), (CS-190)
4	(CS-319), (CS-315), (CS-101), (CS-190)
5	done

Recursion in SQL - Semantics

- Recursive views are required to be **monotonic**.
 - If we add tuples to *prereq* the view *rec_prereq* contains all of the tuples it contained before plus possibly more
- Recursive queries may not use any of the constructs.
 - **Aggregation** on the recursive view
 - **not exists** on a subquery that uses the recursive view
 - Set difference (**except**) whose right-hand side uses the recursive view

The Power of Recursion

- Recursive views make it possible to write queries, such as transitive closure queries, that cannot be written without recursion or iteration.
 - Intuition: Without recursion, a non-recursive non-iterative program can perform only a fixed number of joins of *prereq* with itself
 - Given a fixed non-recursive query, we can construct a database with a greater number of levels of prerequisites on which the query will not work
- Alternative?

The Power of Recursion

- Recursive views make it possible to write queries, such as transitive closure queries, that cannot be written without recursion or iteration.
 - Intuition: Without recursion, a non-recursive non-iterative program can perform only a fixed number of joins of *prereq* with itself
 - Given a fixed non-recursive query, we can construct a database with a greater number of levels of prerequisites on which the query will not work
- Alternative: write a procedure to iterate as many times as required
 - How?

Example Procedure for Iteration

create function *findAllPrereqs*(*cid* **varchar(8)**)

- - Finds all courses that are prerequisite (directly or indirectly) for *cid*

returns table (*course_id* **varchar(8)**)

- - The relation *prereq(course_id, prereq_id)* specifies which course is
- - directly a prerequisite for another course.

begin

create temporary table *c_prereq* (*course_id* **varchar(8)**);

- - table *c_prereq* stores the set of courses to be returned

create temporary table *new_c_prereq* (*course_id* **varchar(8)**);

- - table *new_c_prereq* contains courses found in the previous iteration

create temporary table *temp* (*course_id* **varchar(8)**);

- - table *temp* is used to store intermediate results

insert into *new_c_prereq*

select *prereq_id*

from *prereq*

where *course_id* = *cid*;

Example Procedure for Iteration

```
repeat
    insert into c_prereq
        select course_id
        from new_c_prereq;

    insert into temp
        (select prereq.prereq_id
         from new_c_prereq, prereq
         where new_c_prereq.course_id = prereq.course_id
        )
    except (
        select course_id
        from c_prereq
    );
    delete from new_c_prereq;
    insert into new_c_prereq
        select *
        from temp;
    delete from temp;

until not exists (select * from new_c_prereq)
end repeat;
return table c_prereq;
end
```

Recursion Example

- Given relation
manager(employee_name, manager_name)
- Find all employee-manager pairs, where the employee reports to the manager directly or indirectly (that is manager's manager, manager's manager's manager, etc.)

Recursion Example

- Given relation
manager(employee_name, manager_name)
- Find all employee-manager pairs, where the employee reports to the manager directly or indirectly (that is manager's manager, manager's manager's manager, etc.)
 - **with recursive** *empl* (*employee_name*, *manager_name*) **as** (
 select *employee_name*, *manager_name*
 from *manager*
 union
 select *manager.employee_name*, *empl.manager_name*
 from *manager*, *empl*
 where *manager.manager_name* = *empl.employee_name*)
 select *
 from *empl*

This example view, *empl*, is the *transitive closure* of the *manager* relation

Outline

- Functions and Procedures
- Triggers
- Recursive Queries
- **Advanced Aggregation Features**
- Accessing SQL From a Programming Language

Over Clause

- New type of aggregation and grouping
 - Each input tuple is paired with the aggregation result for the group it belongs to
 - More flexible grouping based on order and windowing
 - New aggregation functions such as ranking
- In the form of
 - *Agg **over** (partition clause, order by, window specification)*

Over Clause

- **select** year, month, city, profit, **sum**(profit) **over**() **as** ttl
from sales
 - For each tuple, build a set of tuples that belong to the same window
 - Compute aggregation function over the window
 - Return each input tuple paired with the aggregation result for its window
 - **over**(): one window containing all tuples

year	month	city	profit	ttl
2010	1	Chicago	10	92
2010	2	Chicago	5	92
2010	3	Chicago	20	92
2011	1	Chicago	45	92
2010	1	New York	12	92

Over Clause

- **select** year, month, city, profit,
 sum(profit) **over**(**partition by** year) **as** ttl
from sales

year	month	city	profit
2010	1	Chicago	10
2010	2	Chicago	5
2010	3	Chicago	20
2011	1	Chicago	45
2010	1	New York	12

Over Clause

- **select** year, month, city, profit,
 sum(profit) **over**(**partition by** year) **as** ttl
from sales
 - **partition by**: tuples with same partition by attributes belong to the same window (such as **group by**)
 - ▶ Difference?

year	month	city	profit	ttl
2010	1	Chicago	10	47
2010	2	Chicago	5	47
2010	3	Chicago	20	47
2011	1	Chicago	45	45
2010	1	New York	12	47

Over Clause

- **select** year, month, city, profit,
 sum(profit) **over**(**partition by** year) **as** ttl
from sales
 - **partition by**: tuples with same partition by attributes belong to the same window (such as **group by**)
 - ▶ Keeping the original values
 - ▶ Allowing us to produce aggregated values

year	month	city	profit	ttl
2010	1	Chicago	10	47
2010	2	Chicago	5	47
2010	3	Chicago	20	47
2011	1	Chicago	45	45
2010	1	New York	12	47

Over Clause

- **select** year, month, city, profit,
 sum(profit) **over**(**order by** year, month) **as** ttl
from sales

year	month	city	profit
2010	1	Chicago	10
2010	2	Chicago	5
2010	3	Chicago	20
2011	1	Chicago	45
2010	1	New York	12

Over Clause

- **select** year, month, city, profit,
 sum(profit) **over**(**order by** year, month) **as** ttl
from sales
 - **order by**: tuples that are less than equal to the order belong to the same window.
 - Can be used to compute accumulated total

year	month	city	profit	ttl
2010	1	Chicago	10	22
2010	2	Chicago	5	27
2010	3	Chicago	20	47
2011	1	Chicago	45	92
2010	1	New York	12	22

Over Clause

- **select** year, month, city, profit,
 sum(profit) **over**(**order by** year, month) **as** ttl
from sales
 - **order by**: tuples that are less than equal to the order belong to the same window.
 - Can be used to compute accumulated total

year	month	city	profit	ttl
2010	1	Chicago	10	22
2010	2	Chicago	5	27
2010	3	Chicago	20	47
2011	1	Chicago	45	92
2010	1	New York	12	22

Over Clause

- **select** year, month, city, profit,
 sum(profit) **over**(**order by** year, month) **as** ttl
from sales
 - **order by**: tuples that are less than equal to the order belong to the same window.
 - Can be used to compute accumulated total

year	month	city	profit	ttl
2010	1	Chicago	10	22
2010	2	Chicago	5	27
2010	3	Chicago	20	47
2011	1	Chicago	45	92
2010	1	New York	12	22

Over Clause

- **select** year, month, city, profit,
 sum(profit) **over**(**order by** year, month) **as** ttl
from sales
 - **order by**: tuples that are less than equal to the order belong to the same window.
 - Can be used to compute accumulated total

year	month	city	profit	ttl
2010	1	Chicago	10	22
2010	2	Chicago	5	27
2010	3	Chicago	20	47
2011	1	Chicago	45	92
2010	1	New York	12	22

Over Clause

- **select** year, month, city, profit,
 sum(profit) **over**(**partition by** year **order by** month) **as** ttl
from sales

year	month	city	profit
2010	1	Chicago	10
2010	2	Chicago	5
2010	3	Chicago	20
2011	1	Chicago	45
2010	1	New York	12

Over Clause

- **select** year, month, city, profit,
 sum(profit) **over**(**partition by** year **order by** month) **as** ttl
from sales

year	month	city	profit	ttl
2010	1	Chicago	10	22
2010	2	Chicago	5	27
2010	3	Chicago	20	47
2011	1	Chicago	45	45
2010	1	New York	12	22

Over Clause

- **select** year, month, city, profit,
 sum(profit) **over**(**partition by** year **order by** month
 range between 1 preceding
 and 1 following) **as** ttl
from sales

year	month	city	profit
2010	1	Chicago	10
2010	2	Chicago	5
2010	3	Chicago	20
2011	1	Chicago	45
2010	1	New York	12

Over Clause

- **select** year, month, city, profit,
 sum(profit) **over**(**partition by** year **order by** month
 range between 1 preceding
 and 1 following) **as** ttl

from sales

year	month	city	profit	ttl
2010	1	Chicago	10	27
2010	2	Chicago	5	47
2010	3	Chicago	20	25
2011	1	Chicago	45	45
2010	1	New York	12	27

Over Clause

- **select** year, month, city, profit,
 sum(profit) **over**(**partition by** year **order by** month
 range between 1 preceding
 and 1 following) **as** ttl

from sales

- Explicit window specification
 - ▶ Determines which tuples “surrounding” the tuple according to the sort order to include in the window

year	month	city	profit	ttl
2010	1	Chicago	10	27
2010	2	Chicago	5	47
2010	3	Chicago	20	25
2011	1	Chicago	45	45
2010	1	New York	12	27

Over Clause

- **select** year, month, city, profit,
 sum(profit) **over**(**order by** year , month
 rows between 1 preceding
 and 1 following) **as** ttl
from sales
 - Explicit window specification
 - ▶ Determines which tuples “surrounding” the tuple according to the sort order to include in the window

year	month	city	profit
2010	1	Chicago	10
2010	2	Chicago	5
2010	3	Chicago	20
2011	1	Chicago	45
2010	1	New York	12

Over Clause

- **select** year, month, city, profit,
 sum(profit) **over**(**order by** year , month
 rows between 1 preceding
 and 1 following) **as** ttl

from sales

- Explicit window specification
 - ▶ Determines which tuples “surrounding” the tuple according to the sort order to include in the window

year	month	city	profit	ttl
2010	1	Chicago	10	22
2010	2	Chicago	5	37
2010	3	Chicago	20	70
2011	1	Chicago	45	65
2010	1	New York	12	27

Windowing

- Window queries compute an aggregate function over ranges of tuples.
 - Useful, e.g., to compute an aggregate of a fixed range of time
 - The **time range** is called a **window**.
- Windows may overlap in which case a tuple may contribute to more than one window.

Windowing

- Used to smooth out random variations.
- E.g., **moving average**: “Given sales values for each date, calculate for each date the average of the sales on that day, the previous day, and the next day”
- **Window specification** in SQL:
 - Given relation *sales(date, value)*

Windowing

- Used to smooth out random variations.
- E.g., **moving average**: “Given sales values for each date, calculate for each date the average of the sales on that day, the previous day, and the next day”
- **Window specification** in SQL:
 - Given relation *sales(date, value)*
select *date*, **avg**(*value*) **over**
 (**order by** *date* **rows between 1 preceding and 1 following**)
from *sales*

Windowing

- Examples of other window specifications:
 - **rows between unbounded preceding and current**
 - **rows unbounded preceding**
 - The window consists of all previous
 - **rows between 10 preceding and current row**
 - All rows with values between current row value -10 to current value
 - **range interval 10 day preceding**
 - Not including current row

Windowing

- Another Example
 - *tot_credits (year, num_credits)*
 - The total number of credits taken by students in each year
 - Contain at most one tuple for each year
 - Compute the moving average credits for the window of 3 previous years
 - Compute the moving average credits for all prior years
 - Compute the moving average credits for the window of 3 previous years and the next 2 years

Windowing

- Another Example

- *tot_credits (year, num_credits)*
 - The total number of credits taken by students in each year
 - Contain at most one tuple for each year
- Compute the moving average credits for the window of 3 previous years

```
select year, avg(num_credits)  
           over (order by year rows 3 preceding)  
           as avg_total_credits  
from tot_credits;
```

Windowing

- Another Example

- *tot_credits (year, num_credits)*
 - The total number of credits taken by students in each year
 - Contain at most one tuple for each year
- Compute the moving average credits for all prior years

```
select year, avg(num_credits)  
          over (order by year rows unbounded preceding)  
          as avg_total_credits  
from tot_credits;
```

Windowing

- Another Example

- *tot_credits (year, num_credits)*
 - The total number of credits taken by students in each year
 - Contain at most one tuple for each year
- Compute the moving average credits for the window of 3 previous years and the next 2 years

```
select year, avg(num_credits)  
          over (order by year rows between 3 preceding and 2 following)  
          as avg_total_credits  
from tot_credits;
```

Windowing (Cont.)

- Can do windowing within partitions
- E.g., Given a relation *transaction* (*account_number*, *date_time*, *value*), where *value* is positive for a deposit and negative for a withdrawal
 - “Find total balance of each account after each transaction on the account”

Windowing (Cont.)

- Can do windowing within partitions
- E.g., Given a relation *transaction* (*account_number*, *date_time*, *value*), where *value* is positive for a deposit and negative for a withdrawal
 - “Find total balance of each account after each transaction on the account”

```
select account_number, date_time,  
       sum (value) over  
         (partition by account_number  
          order by date_time  
          rows unbounded preceding)  
       as balance  
from transaction  
order by account_number, date_time
```

Ranking

- Suppose we are given a relation
student_grades(*ID*, *GPA*)
giving the grade-point average of each student
- Using the SQL constructs we have seen so far
 - Difficult to express
 - Inefficient to evaluate
 - Programmers may write the query partly in SQL and programming language

Ranking

- Suppose we are given a relation
student_grades(*ID*, *GPA*)
giving the grade-point average of each student
- Ranking is done in conjunction with an order by specification.
- Find the rank of each student
 - **select** *ID*, **rank()** **over** (**order by** *GPA* **desc**) **as** *s_rank*
from *student_grades*

Ranking

- Suppose we are given a relation
student_grades(*ID*, *GPA*)
giving the grade-point average of each student
- Ranking is done in conjunction with an order by specification.
- Find the rank of each student
 - **select** *ID*, **rank()** **over** (**order by** *GPA* **desc**) **as** *s_rank*
from *student_grades*
- An extra **order by** clause is needed to get them in sorted order.
 - **select** *ID*, **rank()** **over** (**order by** *GPA* **desc**) **as** *s_rank*
from *student_grades*
order by *s_rank*

Ranking

- Suppose we are given a relation
student_grades(*ID*, *GPA*)
giving the grade-point average of each student
- Ranking is done in conjunction with an order by specification.
- Find the rank of each student
 - **select** *ID*, **rank()** **over** (**order by** *GPA desc*) **as** *s_rank*
from *student_grades*
- An extra **order by** clause is needed to get them in sorted order
 - **select** *ID*, **rank()** **over** (**order by** *GPA desc*) **as** *s_rank*
from *student_grades*
order by *s_rank*
- Ranking may leave gaps: e.g., if 2 students have the same top GPA, both have rank 1, and the next rank is 3

Ranking

- Suppose we are given a relation
student_grades(*ID*, *GPA*)
giving the grade-point average of each student
- Ranking is done in conjunction with an order by specification.
- Find the rank of each student
 - **select** *ID*, **rank()** **over** (**order by** *GPA desc*) **as** *s_rank*
from *student_grades*
- An extra **order by** clause is needed to get them in sorted order
 - **select** *ID*, **rank()** **over** (**order by** *GPA desc*) **as** *s_rank*
from *student_grades*
order by *s_rank*
- Ranking may leave gaps: e.g., if 2 students have the same top GPA, both have rank 1, and the next rank is 3
 - **dense_rank()** does not leave gaps, so next dense rank would be 2

Ranking

- Ranking can be done using basic SQL aggregation, but resultant query is very inefficient.

Ranking

- Ranking can be done using basic SQL aggregation, but resultant query is very inefficient.

```
select ID, (1 + (select count(*)  
                    from student_grades B  
                    where B.GPA > A.GPA)) as s_rank  
from student_grades A  
order by s_rank;
```

- Rank clause can sort the relation and compute the rank in much less time.

Ranking (Cont.)

- Ranking can be done within partition of the data.
- “Find the rank of students within each department.”
 - Schema: (ID, GPA, dept_name)

Ranking (Cont.)

- Ranking can be done within partition of the data.
- “Find the rank of students within each department.”
 - Schema: (ID, GPA, dept_name)
 - **select** *ID, dept_name,*
 rank () over (partition by dept_name order by GPA desc)
 as dept_rank
from dept_grades
order by dept_name, dept_rank;

Ranking (Cont.)

- Ranking can be done within partition of the data.
- “Find the rank of students within each department.”
 - Schema: (ID, GPA, dept_name)
 - **select** *ID, dept_name,*
 rank () over (partition by dept_name order by GPA desc)
 as dept_rank
 from *dept_grades*
 order by *dept_name, dept_rank;*
- Multiple **rank** clauses can occur in a single **select** clause.
 - E.g., overall rank and the rank within the department

Ranking (Cont.)

- Ranking can be done within partition of the data.
- “Find the rank of students within each department.”
 - Schema: (ID, GPA, dept_name)
 - **select** *ID, dept_name,*
 rank () over (partition by dept_name order by GPA desc)
 as dept_rank
 from dept_grades
 order by dept_name, dept_rank;
- Multiple **rank** clauses can occur in a single **select** clause.
 - E.g., overall rank and the rank within the department
- Ranking is done *after* applying **group by** clause/aggregation
 - The aggregate values can be used for ranking.

Ranking (Cont.)

- Can be used to find top-n results
 - More general than the **limit** n clause supported by many databases, since it allows top-n within each partition

Ranking (Cont.)

- Can be used to find top-n results
 - More general than the **limit** *n* clause supported by many databases, since it allows top-n within each partition
 - **select** *ID, GPA*
from *student_grades*
order by *GPA desc*
limit 5;

Ranking (Cont.)

- Can be used to find top-n results
 - More general than the **limit** *n* clause supported by many databases, since it allows top-n within each partition
 - **select** *ID, GPA*
from *student_grades*
order by *GPA desc*
limit 5;
 - **select** *
from (**select** *ID, rank()* **over** (**order by** *GPA desc*) **as** *s_rank*
 from *student_grades*)
where *s_rank* <= 5;

Ranking (Cont.)

- Can be used to find top-n results
 - More general than the **limit** *n* clause supported by many databases, since it allows top-n within each partition
 - **select** *ID, dept_name, GPA*
from *dept_grades*
order by *GPA desc*
limit 5;

Ranking (Cont.)

- Can be used to find top-n results
 - More general than the **limit** *n* clause supported by many databases, since it allows top-n within each partition
 - **select** *ID, dept_name, GPA*
from *dept_grades*
order by *GPA desc*
limit 5;
 - **select** *
from (**select** *ID, dept_name,*
 rank () **over** (**partition by** *dept_name* **order by** *GPA desc*)
 as *dept_rank*
 from *dept_grades*
 order by *dept_name, dept_rank*)
where *dept_rank* <= 5;

Ranking (Cont.)

- Other ranking functions:
 - **percent_rank** (within partition, if partitioning is done)
 - The rank as a fraction defined $(r-1) / (n-1)$
 - n tuples and the rank of the tuple is r
 - **cume_dist** (cumulative distribution)
 - fraction of tuples with preceding values
 - **row_number** (non-deterministic in presence of duplicates)
 - Sorts and gives each row a unique number
 - Different rows with the same ordering value → different row numbers

Ranking (Cont.)

- Null values need to be taken care of while ranking

Ranking (Cont.)

- Null values need to be taken care of while ranking
 - Null values are treated as the highest values
- SQL:1999 permits the user to specify **nulls first** or **nulls last**
select *ID*,
 rank () over (order by *GPA desc nulls last***) as** *s_rank*
from *student_grades*

Ranking (Cont.)

- For a given constant n , the ranking the function **ntile**(n) takes the tuples in each partition in the specified order, and divides them into n buckets with equal numbers of tuples.
 - Particularly useful for constructing histograms
- Example of quartile

```
select ID, ntile(4) over (order by GPA desc) as quartile  
from student_grades;
```

Outline

- Functions and Procedures
- Triggers
- Recursive Queries
- Advanced Aggregation Features
- **Accessing SQL From a Programming Language**

Accessing SQL from a Programming Language

- A database programmer must have access to a general-purpose programming language for at least two reasons.

Accessing SQL from a Programming Language

- A database programmer must have access to a general-purpose programming language for at least two reasons.
 - Not all queries can be expressed in SQL
 - SQL does not provide the full expressive power of a general-purpose language.
 - Non-declarative actions
 - Printing a report, interacting with a user, or sending the results of a query to a graphical user interface -- cannot be done from within SQL.

Accessing SQL from a Programming Language

- **Dynamic SQL** -- can connect to and communicate with a database server using a collection of functions or methods
 - Allows the program to construct an SQL query as a character string at runtime, submit the query, and then retrieve the result into program variables a tuple at a time.
 - JDBC & ODBC
 - Extended to C++, Python, PHP, Go, etc
- **Embedded SQL** -- provides a means by which a program can interact with a database server.
 - The SQL statements are translated at compile time into function calls.
 - At runtime, these function calls connect to the database using an API that provides dynamic SQL facilities.

Accessing SQL from a Programming Language

- A Major challenge is the **mismatch** in the ways that manipulate data
 - SQL operates on relations and returns a relation as a result.
 - Programming language operates on a variable.
 - Variables correspond to roughly the value of an attribute of a tuple.
- Providing mechanism to return the result of a query

JDBC

- **JDBC** is a Java API for communicating with database systems supporting SQL.
- JDBC supports a **variety of features** for querying and updating data, and for retrieving query results.
- JDBC also supports **metadata retrieval**, such as querying about relations present in the database and the names and types of relation attributes.
- Model for communicating with the database:
 - Open a connection
 - Create a “statement” object
 - Execute queries using the statement object to send queries and fetch results
 - Exception mechanism to handle errors

JDBC Code

```
public static void JDBCexample(String dbid, String userid, String passwd)
{
    try {
        Class.forName ("oracle.jdbc.driver.OracleDriver");
        Connection conn = DriverManager.getConnection( // connect to server
            "jdbc:oracle:thin:@db.yale.edu:2000:univdb", userid, passwd);
        Statement stmt = conn.createStatement(); // create Statement object
        ... Do Actual Work ....
        stmt.close(); // close Statement and release resources
        conn.close(); // close Connection and release resources
    }
    catch (SQLException sqle) {
        System.out.println("SQLException : " + sqle); // handle exceptions
    }
}
```

Connecting to the Database

- To open a connection to the database
 - Select which database to use
- Using **getConnection()** method
 - Connection conn = DriverManager.**getConnection**(// connect to server
"jdbc:oracle:thin:@db.yale.edu:2000:univdb", userid, passwd);
 - Specifying the URL
 - ▶ Protocol: jdbc:oracle:thin:
 - ▶ Port: 2000
 - ▶ Database server: rhode.uc.edu
 - ▶ Database: univdb
 - User identifier
 - Password

Shipping SQL Statements to Database System

- Once the connection is open, the program can use an instance of the class **Statement** to send SQL statements to the database system.
- A **Statement** object allows the Java program to invoke methods.
 - `Statement stmt = conn.createStatement();` // create Statement object
- To execute a statement
 - **executeQuery()**: SQL statement is a query
 - **executeUpdate()**: nonquery statement
 - ▶ Update, insert, delete or create

JDBC Code (Cont.)

- Update to database

```
try {  
    stmt.executeUpdate(  
        "insert into instructor values('77987', 'Kim', 'Physics', 98000)");  
} catch (SQLException sqle)  
{  
    System.out.println("Could not insert tuple. " + sqle);  
}
```

- Execute query and fetch and print results

```
ResultSet rs = stmt.executeQuery(  
    "select dept_name, avg (salary)  
    from instructor  
    group by dept_name");  
while (rs.next()) {  
    System.out.println(rs.getString("dept_name") + " " +  
        rs.getFloat(2));  
}
```

JDBC Code Details

- Result stores the current row position in the result
 - Pointing before the first row after executing the statement
 - **.next()** moves to the next tuple
 - Returns false if no more tuples
- Getting result fields:
 - **rs.getString("dept_name")** and **rs.getString(1)** equivalent if dept_name is the first argument of select result.
- Dealing with Null values
 - `int a = rs.getInt("a");`
`if (rs.wasNull()) Systems.out.println("Got null value");`

Exceptions and Resource Management

- To catch any exceptions (error conditions) that arise
 - **SQLException** for SQL specific exception
 - **Exception** for any Java exception
- Example
 - ```
catch (SQLException sqle) {
 System.out.println("SQLException : " + sqle); // handle exceptions
}
```

# JDBC Code

```
public static void JDBCexample(String dbid, String userid, String passwd)
{
 try {
 Class.forName ("oracle.jdbc.driver.OracleDriver");
 Connection conn = DriverManager.getConnection(// connect to server
 "jdbc:oracle:thin:@db.yale.edu:2000:univdb", userid, passwd);
 Statement stmt = conn.createStatement(); // create Statement object
 ... Do Actual Work
 stmt.close(); // close Statement and release resources
 conn.close(); // close Connection and release resources
 }
 catch (SQLException sqle) {
 System.out.println("SQLException : " + sqle); // handle exceptions
 }
}
```

# JDBC Code

```
public static void JDBCexample(String userid, String passwd)
{
```

```
 try (
 Connection conn = DriverManager.getConnection(
 "jdbc:oracle:thin:@db.yale.edu:1521:univdb",
 userid, passwd);
 Statement stmt = conn.createStatement();
) {
```

Automatically closed  
at the end of the try  
block

```
 try {
 stmt.executeUpdate(
 "insert into instructor values('77987','Kim','Physics',98000)");
 }
 catch (SQLException sqle) {
 System.out.println("Could not insert tuple. " + sqle);
 }
 ResultSet rset = stmt.executeQuery(
 "select dept_name, avg (salary) "+
 " from instructor "+
 " group by dept_name");
 while (rset.next()) {
 System.out.println(rset.getString("dept_name") + " " +
 rset.getFloat(2));
 }
 }
 catch (Exception sqle)
 {
 System.out.println("Exception : " + sqle);
 }
}
```

# Prepared Statement

- We can create a prepared statement in which some values are replaced by “?”.
  - Specifying that actual values will be provided later.
- Each time the query is executed (with new values to replace the “?”s), the database system can **reuse** the previously compiled form of the query and apply the new values.

# Prepared Statement

- We can create a prepared statement in which some values are replaced by “?”.
  - Specifying that actual values will be provided later.
- Each time the query is executed (with new values to replace the “?”s), the database system can **reuse** the previously compiled form of the query and apply the new values.
- **PreparedStatement** pStmt = conn.**prepareStatement**(  
"insert into instructor values(?,?,?,?)");  
  
pStmt.setString(1, "88877");  
pStmt.setString(2, "Perry");  
pStmt.setString(3, "Finance");  
pStmt.setInt(4, 125000);  
pStmt.executeUpdate();  

pStmt.setString(1, "88878");  
pStmt.executeUpdate();

# Prepared Statement

- **WARNING:** always use prepared statements when taking an input from the user and adding it to a query
  - **NEVER** create a query by concatenating strings
  - "insert into instructor values(' " + ID + " ', ' " + name + " ', " + " ' + dept name + " ', " ' balance + ')"
  - What if name is "D'Souza"?

# SQL Injection

- Suppose query is constructed using
  - "select \* from instructor where name = ' " + name + " ' "
- Suppose the user, instead of entering a name, enters:
  - X' or 'Y' = 'Y

# SQL Injection

- Suppose query is constructed using
  - "select \* from instructor where name = ' " + name + " ' "
- Suppose the user, instead of entering a name, enters:
  - X' or 'Y' = 'Y
- The resulting statement becomes:
  - "select \* from instructor where name = "" + "X' or 'Y' = 'Y" + """
  - which is:
    - select \* from instructor where name = 'X' or 'Y' = 'Y'
  - The where clause is always true.

# SQL Injection

- Suppose query is constructed using
  - "select \* from instructor where name = ' " + name + " ' "
- Suppose the user, instead of entering a name, enters:
  - X' or 'Y' = 'Y
- The resulting statement becomes:
  - "select \* from instructor where name = "" + "X' or 'Y' = 'Y" + """
  - which is:
    - select \* from instructor where name = 'X' or 'Y' = 'Y'
  - The where clause is always true.
  - Clever malicious users could have even used
    - X'; update instructor set salary = salary + 10000; --

# SQL Injection

- Suppose query is constructed using
  - "select \* from instructor where name = ' " + name + " ' "
- Suppose the user, instead of entering a name, enters:
  - X' or 'Y' = 'Y
- The resulting statement becomes:
  - "select \* from instructor where name = "" + "X' or 'Y' = 'Y" + """
  - which is:
    - select \* from instructor where name = 'X' or 'Y' = 'Y'
  - The where clause is always true.
  - Clever malicious users could have even used
    - X'; update instructor set salary = salary + 10000; --
- Prepared statement internally uses
  - "select \* from instructor where name = 'X\' or \'Y\' = \'Y'
  - **Internally escapes the special characters → Empty result**

# Metadata Features

- A Java program does not include declarations for data stored in the database.
  - Part of the SQL DDL statements
- The java program that uses JDBC must determine the information directly from the database system at runtime.

# Metadata Features

- ResultSet has a method for metadata, **getMetaData()**.
- After executing query to get a ResultSet rs:
  - ResultSetMetaData rsmd = **rs.getMetaData()**;  
for(int i = 1; i <= rsmd.**getColumnCount()**; i++) {  
    System.out.println(rsmd.**getColumnName(i)**);  
    System.out.println(rsmd.**getColumnTypeName(i)**);  
}

# Metadata (Cont)

- **DatabaseMetaData** interface provides a way to find metadata about the database.

- `DatabaseMetaData dbmd = conn.getMetaData();`

```
ResultSet rs = dbmd.getColumns(null, "univdb", "department", "%");
```

```
while(rs.next()) {
```

```
 System.out.println(rs.getString("COLUMN_NAME"),
```

```
 rs.getString("TYPE_NAME");
```

```
}
```

- `getColumns(Catalog, Schema, Table, and Column)`
- Returns: One row for each column; row has a number of attributes such as `COLUMN_NAME`, `TYPE_NAME`

# Metadata (Cont)

- **DatabaseMetaData** interface provides a way to find metadata about the database.
- DatabaseMetaData dbmd = conn.getMetaData();  
ResultSet rs = dbmd.**getTables** ("", "", "%", new String[] {"TABLES"});  
while(rs.next()) {  
    System.out.println(rs.getString("TABLE\_NAME"));  
}
  - The first three parameters are same as **getColumns()**.
  - The fourth parameter can be used to restrict the types of tables returned
    - If null → all tables including system internal tables
    - Set to restrict the tables returned to only user-created tables
  - Returns: One row for each table; row has a number of attributes such as TABLE\_NAME, TABLE\_CAT, TABLE\_TYPE, ..

# Finding Primary Keys

- DatabaseMetaData dmd = connection.getMetaData();

```
// Arguments below are: Catalog, Schema, and Table
// The value "" for Catalog/Schema indicates current catalog/schema
// The value null indicates all catalogs/schemas
```

```
ResultSet rs = dmd.getPrimaryKeys("", "", tableName);
```

```
while(rs.next()){
```

```
 // KEY_SEQ indicates the position of the attribute in
 // the primary key, which is required if a primary key has multiple
 // attributes
```

```
 System.out.println(rs.getString("KEY_SEQ"),
 rs.getString("COLUMN_NAME");
```

```
}
```

# Transaction Control in JDBC

- By default, each SQL statement is treated as a separate transaction that is committed automatically
  - bad idea for transactions with multiple updates
- Can turn off automatic commit on a connection
  - `conn.setAutoCommit(false);`
  - `conn.setAutoCommit(true)` to turn it on
- Transactions must then be committed or rolled back explicitly
  - `conn.commit();`    or
  - `conn.rollback();`

# Other JDBC Features

- Calling functions and procedures
  - `CallableStatement cStmt1 = conn.prepareCall("{? = call some function(?)})");`
  - `CallableStatement cStmt2 = conn.prepareCall("{call some procedure(?,?)})");`
- Handling large object types
  - `getBlob()` and `getClob()` that are similar to the `getString()` method, but return objects of type `Blob` and `Clob`, respectively
  - get data from these objects by `getBytes()` and `getSubString()`
  - associate an open stream with Java `Blob` or `Clob` object to update large objects
    - `blob.setBlob(int parameterIndex, InputStream inputStream).`

# JDBC Resources

- JDBC Basics Tutorial
  - Oracle
    - <https://docs.oracle.com/javase/tutorial/jdbc/index.html>
  - Postgres
    - <https://jdbc.postgresql.org/index.html>
  - ...

# ODBC

- Open DataBase Connectivity (ODBC) standard
  - Standard for application program to communicate with a database server
  - Application program interface (API) to
    - open a connection with a database
    - send queries and updates
    - get back results
- Applications such as GUI, spreadsheets, etc. can use ODBC
- Defined originally for Basic and C, versions available for many languages

# ODBC (Cont.)

- Each database system supporting ODBC provides a "**driver**" library that must be linked with the client program.
- When client program makes an ODBC API call, the code in the library communicates with the server to carry out the requested action, and fetch results.
- Opens database connection using **SQLConnect()**. Parameters for SQLConnect:
  - connection handle
  - the server to which to connect
  - the user identifier
  - password
- Must also specify types of arguments:
  - SQL\_NTS denotes previous argument is a null-terminated string.

# ODBC Code

```
■ int ODBCexample()
{
 RETCODE error;
 HENV env; /* environment */
 HDBC conn; /* database connection */
 SQLAllocEnv(&env);
 SQLAllocConnect(env, &conn);
 SQLConnect(conn, "db.yale.edu", SQL_NTS, "avi", SQL_NTS, "avipasswd",
 SQL_NTS);
 { Do actual work ... }

 SQLDisconnect(conn);
 SQLFreeConnect(conn);
 SQLFreeEnv(env);
}
```

# ODBC Code (Cont.)

- Program sends SQL commands to database by using **SQLExecDirect**
- Result tuples are fetched using **SQLFetch()**
- **SQLBindCol()** binds C language variables to attributes of the query result
  - When a tuple is fetched, its attribute values are automatically stored in corresponding C variables.
  - Arguments to **SQLBindCol()**
    - ODBC stmt variable, attribute position in query result
    - The type conversion from SQL to C.
    - The address of the variable.
    - For variable-length types like character arrays,
      - The maximum length of the variable
      - Location to store actual length when a tuple is fetched.
      - Note: A negative value returned for the length field indicates null value

## ODBC Code (Cont.)

- Main body of program

```
char deptname[80];
float salary;
int lenOut1, lenOut2;
HSTMT stmt;
char * sqlquery = "select dept_name, sum (salary)
 from instructor
 group by dept_name";
SQLAllocStmt(conn, &stmt);
error = SQLExecDirect(stmt, sqlquery, SQL_NTS);
if (error == SQL_SUCCESS) {
 SQLBindCol(stmt, 1, SQL_C_CHAR, deptname , 80, &lenOut1);
 SQLBindCol(stmt, 2, SQL_C_FLOAT, &salary, 0 , &lenOut2);
 while (SQLFetch(stmt) == SQL_SUCCESS) {
 printf (" %s %g\n", deptname, salary);
 }
}
SQLFreeStmt(stmt, SQL_DROP);
```

# ODBC Prepared Statements

- **Prepared Statement**

- SQL statement prepared: compiled at the database
- Can have placeholders: E.g. insert into account values(?,?,?)
- Repeatedly executed with actual values for the placeholders

- To prepare a statement

`SQLPrepare(stmt, <SQL String>);`

- To bind parameters

`SQLBindParameter(stmt, <parameter#>, ... type information and value omitted for simplicity..)`

- To execute the statement

`SQLExecute(stmt);`

- To avoid SQL injection security risk, do not create SQL strings directly using user input; instead use prepared statements to bind user inputs

# More ODBC Features

- **Metadata features**

- finding all the relations in the database and
- finding the names and types of columns of a query result or a relation in the database.

- By default, each SQL statement is treated as a separate transaction that is committed automatically.

- Can turn off automatic commit on a connection
  - ▶ `SQLSetConnectOption(conn, SQL_AUTOCOMMIT, 0)`
- Transactions must then be committed or rolled back explicitly by
  - ▶ `SQLTransact(conn, SQL_COMMIT)` or
  - ▶ `SQLTransact(conn, SQL_ROLLBACK)`

# Embedded SQL

- The SQL standard defines embeddings of SQL in a variety of programming languages such as C, C++, Java, Fortran, and PL/I
- A language to which SQL queries are embedded is referred to as a **host language**, and the SQL structures permitted in the host language comprise *embedded SQL*.
- **EXEC SQL** statement is used in the host language to identify embedded SQL request to the preprocessor

EXEC SQL <embedded SQL statement >;

Note: this varies by language:

- In some languages, like COBOL, the semicolon is replaced with END-EXEC
- In Java embedding uses `# SQL { .... };`

## Embedded SQL (Cont.)

- Before executing any SQL statements, the program must first connect to the database.

EXEC SQL **connect to** *server* **user** *user-name* **using** *password*;

- Variables of the host language can be used within embedded SQL statements. They are preceded by a **colon** (:) to distinguish from SQL variables (e.g., *:credit\_amount* )
- **Variables** used as above must be **declared** within DECLARE section, as illustrated below. The syntax for declaring the variables, however, follows the usual host language syntax.

EXEC SQL BEGIN DECLARE SECTION;

int *credit-amount* ;

EXEC SQL END DECLARE SECTION;

# Embedded SQL (Cont.)

- To write an embedded SQL query, we use
  - declare *c* cursor for <SQL query>**
  - The variable *c* is used to identify the query
- Example:
  - From within a host language, find the ID and name of students who have completed more than the number of credits stored in variable **credit\_amount** in the host language
  - Specify the query in SQL as follows:

EXEC SQL

```
declare c cursor for
select ID, name
from student
where tot_cred > :credit_amount;
```

# Embedded SQL (Cont.)

- The **open** statement for our example is as follows:

**EXEC SQL open c ;**

- This statement causes the database system to **execute** the query and to **save** the results within a temporary relation.
  - The query uses the value of the host-language variable *credit-amount* at the time the **open** statement is executed.
- The **fetch** statement causes the values of one tuple in the query result to be placed on host language variables.

**EXEC SQL fetch c into :sj, :sn;**

- Single fetch returns only one tuple.
- Repeated calls to fetch get successive tuples in the query result

## Embedded SQL (Cont.)

- The **close** statement causes the database system to delete the temporary relation that holds the result of the query.

**EXEC SQL close c ;**

# Updates Through Embedded SQL

- Embedded SQL expressions for database modification
  - **update**, **insert**, and **delete**
- Can update tuples fetched by cursor by declaring that the cursor is for update

EXEC SQL

```
declare c cursor for
select *
from instructor
where dept_name = 'Music'
for update;
```

- We then iterate through the tuples by performing **fetch** operations on the cursor (as illustrated earlier).

EXEC SQL

```
update instructor
set salary = salary + 100
where current of c;
```

# Embedded SQL - Example

```
/* define a cursor */
EXEC SQL DECLARE emp_cursor CURSOR FOR
 SELECT ename, job
 FROM emp
 WHERE empno = :emp_number
 FOR UPDATE OF job;

/* open the cursor and identify the active set */
EXEC SQL OPEN emp_cursor;

/* break if the last row was already fetched */
EXEC SQL WHENEVER NOT FOUND DO break;

/* fetch and process data in a loop */
for (;;)
{
 EXEC SQL FETCH emp_cursor INTO :emp_name, :job_title;

/* optional host-language statements that operate on
the FETCHed data */

 EXEC SQL UPDATE emp
 SET job = :new_job_title
 WHERE CURRENT OF emp_cursor;
}
```

# SQLJ

- SQLJ: embedded SQL in Java
  - **#sql** iterator deptInfolter (String dept name, int avgSal);  
deptInfolter iter = null;  
**#sql** iter = { select dept\_name, avg(salary) from instructor  
group by dept name };  
while (**iter.next()**) {  
    String deptName = iter.dept\_name();  
    int avgSal = iter.avgSal();  
    System.out.println(deptName + " " + avgSal);  
}  
iter.close();

# Corresponding Reading Materials

- Database System Concepts 6<sup>th</sup> & 7<sup>th</sup> Edition
  - Chapter 5